AuraChat

Task Planning Document

SDLC Stage 3 — Broken down for AI-assisted (Vibe) Coding

Version 1.0 | March 2026 | Author: Patrick

How to Use This Document

Each task in this plan is designed to be given to an AI coding agent as a single, focused prompt. This is the key principle from the SDLC video: AI agents perform significantly better on small, well-scoped tasks than on large open-ended ones.

➤ *Before giving any task to your AI agent, always paste: (1) the relevant section of the TDD, (2) the task description from this document, and (3) any files the agent needs to be aware of.*

Task Table Columns Explained

- ID — Unique task identifier. P1 = Phase 1, P2 = Phase 2, etc.
- Task — The name of the task and what to build.
- Context to Give AI Agent — What extra information to include in your prompt so the agent has enough context.
- Expected Output — What file(s) or result you should have after this task is done.
- Est. — Rough time estimate for a vibe coding session on this task.

Phase Summary

Phase	Name	What Gets Built	Tasks	Total Est.
P1	**Project Foundation**	Android Studio project, Gradle, Hilt setup, Theme	5 tasks	**~2 hrs**
P2	**Data Layer**	Room DB, DAOs, entities, DatabaseModule	5 tasks	**~2 hrs**
P3	**Firebase & AI**	Firebase setup, GeminiDataSource, streaming Flow	4 tasks	**~2 hrs**

Phase	Name	What Gets Built	Tasks	Total Est.
P4	**Repository & Domain**	ChatRepository, Use Cases, RepositoryModule	4 tasks	~1.5 hrs
P5	**Home Screen**	HomeScreen UI, HomeViewModel, navigation trigger	4 tasks	~2 hrs
P6	**Chat Screen**	ChatScreen UI, streaming bubbles, ChatViewModel	6 tasks	~3 hrs
P7	**Drawer & History**	Drawer UI, HistoryViewModel, search, delete	4 tasks	~2 hrs
P8	**Settings Screen**	SettingsScreen, SettingsViewModel, DataStore	3 tasks	~1.5 hrs
P9	**Polish & Integration**	Error states, animations, full app wiring	4 tasks	~2 hrs

➤ *Total estimated vibe coding time: ~18 hours across 39 tasks. Complete each phase fully before moving to the next.*

PHASE 1 Project Foundation

Set up Android Studio project, all Gradle dependencies, Hilt application wiring, and the AuraChat dark theme. Everything else depends on this phase being correct.

ID	Task	Context to Give AI Agent	Expected Output	Est.
P1-T1	**Create Android Studio Project**	Paste the Gradle config from the PRD. Kotlin DSL. Package: com.aurachat. minSdk 26, compileSdk 36.	New Android project with correct build.gradle.kts files and no build errors.	20 min
P1-T2	**Add All Gradle Dependencies**	Paste the complete libs.versions.toml and app build.gradle.kts from the PRD Gradle Setup section.	All dependencies synced. Project builds successfully with no missing imports.	20 min
P1-T3	**Create Package Structure**	Paste the package tree from TDD Section 2.2. Create all empty packages under com.aurachat.	All packages exist: data/, domain/, presentation/, ui/ with correct sub-packages.	10 min
P1-T4	**Set Up Hilt Application Class**	Paste TDD Section 3. Create AuraChatApplication.kt with @HiltAndroidApp. Register in AndroidManifest. Annotate MainActivity with @AndroidEntryPoint.	App compiles with Hilt. No 'Hilt component not found' errors.	20 min
P1-T5	**Implement AuraChat Dark Theme**	Color palette from PRD Section 5.2: background #000000, surface #1A1A1A, primary #1A73E8. Create Color.kt, Type.kt, Theme.kt. Apply theme in MainActivity.	App launches with pure black background. Material3 theme active.	30 min

➤ *Checkpoint P1: Build and run the app. You should see a black screen with no crashes. If Hilt fails, check @HiltAndroidApp on Application and @AndroidEntryPoint on MainActivity.*

PHASE 2 Data Layer — Room Database

Build the local database that stores all chat sessions and messages. This is the persistence foundation of AuraChat.

ID	Task	Context to Give AI Agent	Expected Output	Est.
P2-T1	**Create ChatSession Entity**	Paste TDD Section 4.1 ChatSession schema. Fields: id (UUID String PK), title, createdAt, updatedAt (both Long).	ChatSession.kt entity class with correct @Entity, @PrimaryKey annotations.	15 min
P2-T2	**Create ChatMessage Entity**	Paste TDD Section 4.1 ChatMessage schema. Fields: id, sessionId (FK), role, content, imageUri?, timestamp, isStreaming.	ChatMessage.kt entity with @ForeignKey CASCADE delete on sessionId.	20 min
P2-T3	**Create ChatSessionDao**	Paste TDD Section 4.2 ChatSessionDao methods. getAllSessions() returns Flow. All write ops are suspend functions.	ChatSessionDao.kt interface with correct @Dao, @Query, @Insert, @Update, @Delete.	20 min
P2-T4	**Create ChatMessageDao**	Paste TDD Section 4.2 ChatMessageDao methods. getMessagesForSession() returns Flow ordered by timestamp ASC.	ChatMessageDao.kt interface with all methods correctly annotated.	20 min
P2-T5	**Create Room Database & DatabaseModule**	Paste TDD Section 3.2 DatabaseModule. Create AuraChatDatabase extending RoomDatabase. Create Hilt DatabaseModule providing DB and both DAOs as @Singleton.	AuraChatDatabase.kt and DatabaseModule.kt. App still builds with no Hilt errors.	25 min

➤ *Checkpoint P2: Write a quick unit test or log statement to confirm Room DB is created on first launch. Check Logcat for 'Room database created' or similar.*

PHASE 3 Firebase AI & Gemini Streaming

Connect AuraChat to Firebase AI Logic and implement the streaming response flow. This is the heart of the app.

ID	Task	Context to Give AI Agent	Expected Output	Est.
P3-T1	**Firebase Project Setup**	Create Firebase project at console.firebase.google.com.	google-services.json in place. App syncs	30 min

ID	Task	Context to Give AI Agent	Expected Output	Est.
		Add Android app with package com.aurachat. Enable Gemini API. Download google-services.json to app/ folder. Add to .gitignore.	without Google Services errors.	
P3-T2	**Create FirebaseAIDataSource**	Paste TDD Section 4.3 full spec. streamResponse(history, prompt, imageBitmap?): Flow<String>. Use Firebase.ai(GenerativeBackend.googleAI()). Map each token to String. Catch errors.	FirebaseAIDataSource.kt in data/remote/. @Singleton @Inject constructor.	40 min
P3-T3	**Create AIModule (Hilt)**	Paste TDD Section 3.2 AIModule spec. Provides GenerativeModel as @Singleton using 'gemini-2.0-flash'. Install in SingletonComponent.	AIModule.kt in di/ package. Hilt compiles with no missing binding errors.	20 min
P3-T4	**Test Streaming in Isolation**	Create a temporary test button in MainActivity that calls FirebaseAIDataSource.streamResponse() directly and logs each token to Logcat.	Logcat shows Gemini tokens streaming in one-by-one. Remove test code after.	30 min

➤ *Checkpoint P3: This is the most critical checkpoint. Do NOT proceed to Phase 4 until you can see streaming tokens in Logcat. If Firebase fails, check google-services.json placement and that Gemini API is enabled in Firebase console.*

PHASE 4 Repository & Domain Layer

Wire the data sources together behind a clean interface. ViewModels will only ever talk to this layer.

ID	Task	Context to Give AI Agent	Expected Output	Est.
P4-T1	**Create Domain Models & ChatRepository Interface**	Paste TDD Section 4.4 interface spec. Create ChatRepository interface in domain/repository/. All methods listed in TDD: getSessions, getMessages, createSession, deleteSession, streamAIResponse, saveMessage, updateSessionTitle.	ChatRepository.kt interface in domain layer. Pure Kotlin — no Android imports.	20 min
P4-T2	**Create ChatRepositoryImpl**	Paste TDD Section 4.4 ChatRepositoryImpl spec. Implements ChatRepository. Injects ChatSessionDao, ChatMessageDao, FirebaseAIDataSource. Auto-generates session title from first user message.	ChatRepositoryImpl.kt in data/repository/. All interface methods implemented.	40 min

ID	Task	Context to Give AI Agent	Expected Output	Est.
P4-T3	**Create RepositoryModule (Hilt)**	Paste TDD Section 3.2 RepositoryModule. Use @Binds to bind ChatRepository → ChatRepositoryImpl. Install in SingletonComponent. Abstract module.	RepositoryModule.kt. Hilt dependency graph fully connected. App builds clean.	20 min
P4-T4	**Create Use Cases**	Paste TDD Section 5.1 use case table. Create: GetSessionsUseCase, GetMessagesUseCase, CreateSessionUseCase, DeleteSessionUseCase, SendMessageUseCase. Each is a class with @Inject constructor and single invoke() function.	5 use case files in domain/usecase/. All inject ChatRepository via constructor.	30 min

➤ *Checkpoint P4: The full data pipeline is now wired. Hilt should compile with zero errors. All 3 Hilt modules (Database, AI, Repository) are active.*

PHASE 5 Home Screen

Build the entry point of AuraChat — the greeting screen with suggestion chips and the input bar that starts a new chat.

ID	Task	Context to Give AI Agent	Expected Output	Est.
P5-T1	**Create HomeUiState & HomeViewModel**	Paste TDD Section 6.1 HomeUiState and 6.2 HomeViewModel spec. ViewModel creates a new session via CreateSessionUseCase and emits NavigationEvent.ToChatScreen(sessionId) via SharedFlow.	HomeViewModel.kt with @HiltViewModel. Emits navigation events correctly.	25 min
P5-T2	**Build InputBar Composable**	Shared component used on both HomeScreen and ChatScreen. Contains: TextField (black bg, hint 'Ask AuraChat...'), + attachment button, send IconButton. Callbacks: onSendMessage(String), onAttachImage().	InputBar.kt in ui/components/. Renders correctly in Preview.	30 min
P5-T3	**Build HomeScreen Composable**	Paste UI spec from PRD Section 3.2 F1. Greeting text 'Hi Patrick', 4 suggestion chips, InputBar at bottom. TopBar with hamburger + title 'AuraChat'. Pure black background.	HomeScreen.kt in presentation/home/. Matches Gemini app screenshot aesthetic.	35 min
P5-T4	**Wire HomeScreen Navigation**	Set up NavHost in MainActivity with home as startDestination. Collect HomeViewModel	Tapping send on HomeScreen navigates to ChatScreen	25 min

ID	Task	Context to Give AI Agent	Expected Output	Est.
		navigation SharedFlow in HomeScreen. Navigate to chat/{sessionId} on send or chip tap.	(placeholder OK for now).	

➤ *Checkpoint P5: Launch the app. You should see the HomeScreen with black background, greeting, suggestion chips, and a working input bar. Tapping send should navigate away from HomeScreen.*

PHASE 6 Chat Screen — Core Feature

The most complex phase. Build the chat UI with real-time streaming, message bubbles, and image attachment. Take this one task at a time.

ID	Task	Context to Give AI Agent	Expected Output	Est.
P6-T1	**Create ChatUiState & ChatViewModel (skeleton)**	Paste TDD Section 6.1 ChatUiState and 6.2 ChatViewModel spec. Create the ViewModel with uiState StateFlow. Implement loadMessages(sessionId) on init. No sending yet.	ChatViewModel.kt with @HiltViewModel. Loads messages from Room on init.	**30 min**
P6-T2	**Build MessageBubble Composable**	Paste TDD Section 6.3. USER messages: right-aligned, primary blue bg, white text. MODEL messages: left-aligned, surface #1A1A1A bg, white text. Show timestamp below each bubble.	MessageBubble.kt in ui/components/. Renders both USER and MODEL variants in Preview.	**35 min**
P6-T3	**Build ChatScreen UI (static)**	LazyColumn of MessageBubble items from uiState.messages. InputBar at bottom. TopBar with back arrow + session title. Auto-scroll to bottom on new message. No streaming yet.	ChatScreen.kt renders message history correctly. Auto-scroll works.	**35 min**
P6-T4	**Implement Message Sending & Streaming**	Paste TDD Section 7.1 full 16-step data flow. In ChatViewModel: onSendMessage() adds USER message, collects Flow<String> from SendMessageUseCase, appends each token to currentStreamingText in uiState.	Typing a message and sending shows it appear + AI response streams in token by token.	**50 min**
P6-T5	**Add Markdown Rendering to AI Bubbles**	AI (MODEL) messages use MarkdownText composable from compose-markdown library instead of plain Text. Pass uiState message content to MarkdownText.	Bold, italic, inline code, and code blocks render correctly in AI responses.	**25 min**
P6-T6	**Implement Image Attachment**	Paste TDD Section 7.3 image flow. Add photo picker launcher	User can attach a photo. It previews	**40 min**

ID	Task	Context to Give AI Agent	Expected Output	Est.
		to ChatScreen. Decode URI to Bitmap in ViewModel on IO dispatcher. Show ImagePreview composable above InputBar. Pass Bitmap to SendMessageUseCase on send.	above input. Sends to Gemini with the message.	

➤ *Checkpoint P6: This is the full core loop. Send a text message → see streaming response. Send an image + question → get a vision-based answer. If streaming isn't smooth, revisit FirebaseAIDataSource token mapping.*

PHASE 7 Drawer & Chat History

Build the side drawer that gives access to past conversations. This makes AuraChat feel like a complete, usable app.

ID	Task	Context to Give AI Agent	Expected Output	Est.
P7-T1	**Create HistoryUiState & HistoryViewModel**	Paste TDD Section 6.1 HistoryUiState and 6.2 HistoryViewModel. Collects GetSessionsUseCase flow. Handles searchQuery filtering: filter sessions list where title.contains(query, ignoreCase=true).	HistoryViewModel.kt with search filtering working correctly.	25 min
P7-T2	**Build SessionListItem & DrawerContent Composables**	DrawerContent: search bar at top, 'New Chat' button, LazyColumn of SessionListItem. Each item shows session title (truncated) + relative timestamp (e.g. 'Yesterday', '2 days ago'). Callbacks: onSessionClick(id), onNewChat().	DrawerContent.kt renders session list correctly with search filtering.	40 min
P7-T3	**Wire Drawer into Navigation**	Wrap NavHost in ModalNavigationDrawer in MainActivity. Drawer state hoisted to MainActivity. Hamburger icon opens drawer. Tapping a session navigates to chat/{sessionId}. 'New Chat' calls CreateSessionUseCase and navigates.	Drawer opens/closes. Tapping a past session loads it in ChatScreen.	30 min
P7-T4	**Implement Swipe-to-Delete Sessions**	Add SwipeToDismiss on each SessionListItem in DrawerContent. On swipe: show confirmation snackbar 'Chat deleted'. Call DeleteSessionUseCase. Room Flow auto-updates the list.	Swiping a session item deletes it and removes it from the drawer list.	30 min

➤ *Checkpoint P7: Open the drawer. Past sessions appear. Search filters them. Tap one to resume it. Swipe to delete. New Chat button starts a fresh session.*

PHASE 8 Settings Screen

Allow the user to choose their Gemini model and view app information.

ID	Task	Context to Give AI Agent	Expected Output	Est.
P8-T1	**Set Up DataStore & SettingsRepository**	Create SettingsRepository interface and DataStoreSettingsRepository. Store selected model name as a Preferences DataStore key. Default = 'gemini-2.0-flash'. Add SettingsModule to Hilt.	SettingsRepository with get/set for model preference. DataStore persists across sessions.	**30 min**
P8-T2	**Create SettingsViewModel & SettingsScreen**	SettingsViewModel reads/writes model preference via SettingsRepository. SettingsScreen: model picker (RadioButton group for 3 models), About section (app version, GitHub link).	SettingsScreen.kt navigable from drawer. Model selection persists after app restart.	**35 min**
P8-T3	**Connect Model Selection to AIModule**	Update AIModule to read the selected model from SettingsRepository when providing GenerativeModel. Ensure model name change in settings is reflected in next AI call.	Switching model in settings changes which Gemini model responds in chat.	**25 min**

➤ *Checkpoint P8: Open Settings from drawer. Select a different model. Start a new chat. Confirm the selected model is being used (check Firebase console logs or response behavior).*

PHASE 9 Polish, Error Handling & Integration

Add the finishing touches that separate a working prototype from a portfolio-quality app.

ID	Task	Context to Give AI Agent	Expected Output	Est.
P9-T1	**Implement Error States**	Paste TDD Section 10. Add error banner below message list in ChatScreen when uiState.error is non-null. Include Retry button. Handle: network errors, empty responses, image decode failures.	Error banner appears on API failure. Retry button re-sends last message.	**35 min**
P9-T2	**Add Loading & Typing Indicator**	Show animated typing indicator (3 pulsing dots) in a MODEL bubble while isLoading=true and before first streaming token	Smooth typing indicator appears between send and first token. Disappears when streaming starts.	**30 min**

ID	Task	Context to Give AI Agent	Expected Output	Est.
		arrives. Use Compose animate* APIs.		
P9-T3	**Add Screen Transition Animations**	Add slide-in/slide-out transitions between HomeScreen and ChatScreen using AnimatedNavHost or Compose animation APIs. Drawer open/close uses default Material sliding animation.	Navigating between screens feels smooth and polished.	**25 min**
P9-T4	**Final Integration Review**	Paste TDD Section 11.2 Golden Rules. Ask AI agent to review the full codebase for: logic in Composables, missing error handling, Hilt wiring gaps, hardcoded strings that should be constants.	Code review report with any issues fixed. App passes all 8 success criteria from PRD Section 7.	**45 min**

➤ *Checkpoint P9 — Final: Run through all 8 PRD success criteria. Test on a physical device if possible. AuraChat v1 is complete.*

Quick Reference: Prompt Template

Copy and paste this template every time you start a new task with your AI coding agent:

```
I am building AuraChat — an Android Jetpack Compose AI chat app using Gemini
via Firebase AI Logic, Room, Hilt, and Clean MVVM architecture.
Current task: [PASTE TASK NAME AND DESCRIPTION FROM THIS DOCUMENT]
Relevant architecture spec: [PASTE RELEVANT TDD SECTION]
Rules: No logic in Composables. Use KSP not kapt. All classes use @Inject
constructor. Follow the package structure exactly.
```